

handwriting_analysis

Setup guide — OCR pipeline for handwritten CRF / weekly PT session forms

What this is

A Python pipeline that walks a folder of participant sub-folders (VPT001, VPT002, ...), finds every CRF / weekly-session PDF, and reads the handwritten answers using a hybrid of Tesseract (printed text, checkboxes, circled Yes/No answers) and Qwen2.5-VL, a vision-language model, for the actual handwriting. It produces one markdown report per participant and a combined CSV for stats.

Requirements

- macOS with **Homebrew** installed (developed and tested on an Apple Silicon M2 Pro)
- **Python 3.11** (a separate interpreter is fine even if your default python3 is newer or older)
- **~15 GB free disk space** — the Qwen2.5-VL model is a one-time ~7 GB download, plus the Python environment and its dependencies
- A free **Kaggle account** — only needed for the optional real-handwriting benchmark tools, not for running the pipeline itself

1. Install system dependencies

Tesseract is the OCR engine used for all printed text, checkboxes, and layout detection:

```
brew install tesseract
```

2. Create the virtual environment

From the project folder:

```
cd ~/Desktop/handwriting_analysis
/usr/local/bin/python3.11 -m venv .venv
./venv/bin/pip install --upgrade pip
./venv/bin/pip install -r requirements.txt
```

If `/usr/local/bin/python3.11` doesn't exist on your machine, install it first (`brew install python@3.11`) or point the command above at wherever your Python 3.11 lives.

3. First run — verify it works

Run against the bundled synthetic sample data first (no real participant data needed). This will also trigger the one-time Qwen2.5-VL download (~7 GB, cached under ~/.cache/huggingface afterward):

```
./venv/bin/python src/run_analysis.py
```

This writes into outputs/:

- reports/VPT001_report.md (one per participant) — intake, every weekly session, final visit, and exit survey, with pain/adherence/mileage in a table
- summary.csv — one row per participant-week, ready for Excel/Python/R
- rendered_pages/ and ocr_cache/ — intermediate renders and cached OCR results, so re-runs are fast

4. Point it at the real USB drive

```
./venv/bin/python src/run_analysis.py \  
--base-dir /Volumes/USB --out-dir outputs_real
```

--base-dir just needs a folder containing VPT001/, VPT002/, ... sub-folders in the same layout as the drive. Non-PDF folders (gait-report exports, videos) are ignored automatically.

5. Add keywords (optional, anytime)

Edit config/keywords.txt — one phrase per line, # comments allowed. Every future run automatically searches for them across each participant's OCR'd text and lists matches with context in that participant's report, plus a combined outputs/keyword_hits.csv.

Command reference

Flag	What it does
--base-dir PATH	Folder with VPT001, VPT002, ... sub-folders (default: sample_usb/)
--out-dir PATH	Where reports/CSVs/cache are written (default: outputs/)
--keywords-file PATH	Override the keyword list (default: config/keywords.txt)
--model NAME	Use a different VLM, e.g. Qwen/Qwen2.5-VL-7B-Instruct
--no-vlm	Skip the handwriting model; printed/checkbox/circled fields only (fast)
--participants ID ...	Only process specific participants, e.g. VPT001 VPT003
--force	Ignore the OCR cache and redo everything

Troubleshooting

Disk space is tight

Qwen2.5-VL (~7 GB) plus PyTorch and other dependencies need roughly 10 GB free for the first run. Free up space before running if you're close to the limit — a partially-downloaded model can leave a corrupted cache entry.

A run is taking a long time

The handwriting model reads each field individually, so a full run across many participants takes several minutes (roughly 8 minutes for 5 participants' worth of weekly forms on an M2 Pro). Use `--no-vlm` for a near-instant structural-only pass, or `--participants VPT001` to test on just one participant first.

Regenerating the sample data

```
./venv/bin/python src/generate_sample_data.py
```

Optional: benchmarking against real handwriting

The `benchmark/` folder has tools that download real, labeled handwriting samples (the IAM dataset, via Kaggle) and measure accuracy directly, separate from this project's synthetic sample data. See `README.md` for details and results — a Kaggle account with an API token at `~/.kaggle/kaggle.json` is required for that part only.

See `README.md` in the project folder for the full architecture explanation, model-choice rationale, and known limitations.